COMP90077 Advanced Algorithms and Data Structures

Lecture 1

Overview, Administrivia, Minimum Spanning Trees

Overview

- Combinatorial optimization problems are everywhere
- This subject: general mathematical techniques
- Part 1: Greedy
  - Matroids: Greed is good
  - Submodular functions: Greed is approximately good
- Part 2: Linear Programming
  - Relax and round
  - Duality
- Part 3: Online Algorithms
  - Optimization under uncertainty
- Personal journey

Overview

- Focus on algorithmic ideas
- No coding
- All assignments are written
- Algorithms described using pseudocode
- Prove correctness using mathematical analysis

Administrivia

- Subject coordinator and lecturer: William Umboh
- Head tutor: Xiao Sun

- Course website: https://williamumboh.com/comp90077-2026
- Lecture notes and slides posted before lecture
- Slides meant as presentation aid, not as study material
- Use lecture notes and readings as study material

- Demo website

Expectations

- Time commitment: 200 hours
- 10 – 12 hours per week
- 7 – 9 hours self-study:
  - 3 hours lecture revision
  - 3 hours on tutorial sheet prior to tutorial
  - 3 hours to consolidate after tutorial

Tutorials

- Subject aim: teach problem-solving skills
- Only way to master skill is to practice it

- Tutorial sheet released after lecture
- Expected to spend several hours on problems per week
- Work together with peers before tutorial

- During tutorial, discuss your attempts in groups. Tutor will walk around to provide feedback and/or suggestions

- Solutions released Tuesday 12 pm

Assessments

- 2 written assignments + 1 MST + 1 final exam
- MST:
  - 30 minutes in-class during week 7 lecture
  - 1 handwritten single-sided A4 cheat sheet
- Final exam:
  - 3 hours, 1 handwritten double-sided A4 cheat sheet
- Assignments:
  - Creative problem solving (algorithm design + analysis) + reflection
  - Can pair with up to 1 person

AI Use Statement

GenAI allowed to understand assignment but what you submit must be in own words

Reflection

- Self-evaluation of submitted algorithm and analysis
- Thinking process
- Use of GenAI must be acknowledged in reflection

- Aim 1: Assess critical thinking
- Aim 2: Reward honesty and awareness of the drawbacks of submission

Getting Help

- Office hours:
  - 3 – 4 pm Fridays starting in W2
  - Melbourne Connect 2209-Ngi-a djerring gat-ith (Open Project Space)
- Ed discussion forum
  - Staff will check on daily basis
  - Encouraged to jump in and help others
  - Teaching others is the best way to learn!
- Other resources (see Administrivia on website)

Active Learning

- I will ask questions often to make sure everyone is following
- Interrupt if anything is unclear or to share idea
  - No stupid questions/ideas!
  - Many creative ideas from students
  - Partial red/green blindness
- Bring scratch paper
- Bounce ideas with neighbors

Get to know your classmates

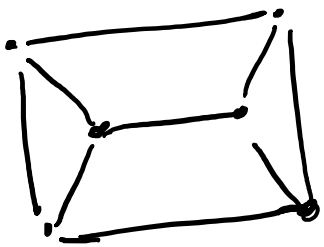
- Name
- Find as many common hobbies as you can with other classmates

Today’s Learning Aims

- Recall graph background
- Recall Minimum Spanning Tree problem
- Prove correctness of Kruskal’s Algorithm
- Analyze time complexity of Kruskal’s Algorithm

Graph Background

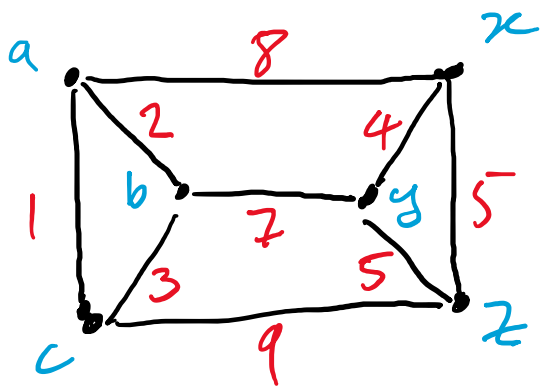
- G = (V, E); V = vertex set, E = edge set
- n = |V|, m = |E|
- Let F be subset of edges
- Key properties:
  - Spanning
  - Connected
  - Acyclic



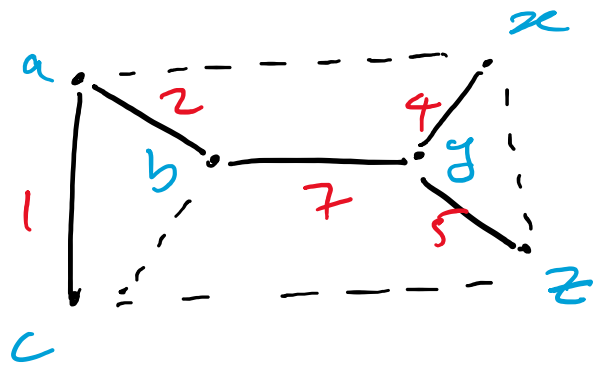
Minimum Spanning Tree (MST)

- Input: G = (V, E), edge weights w_e for each edge e
- Goal: Find spanning subgraph F with cheapest total weight $\sum_{e \in F} w_e$

G



MST:

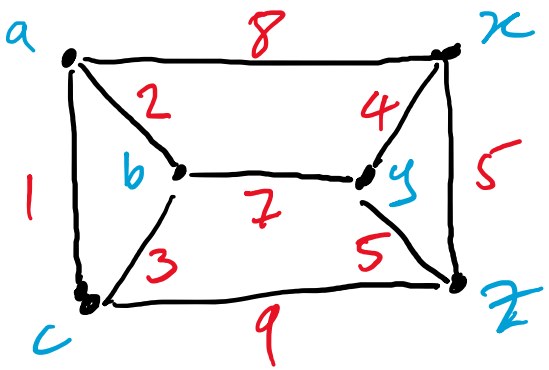


- Discuss: Prove that the optimal F does not have cycles, i.e. is a spanning tree.

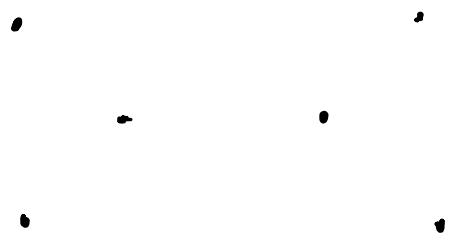
Kruskal's Algorithm

- Initialize S = E, sorted in increasing order of weight
- Initialize F = empty set
- While F is not spanning:
  - Remove min-weight edge e from S
  - Add e to F if doing so does not create cycles and discard otherwise

G

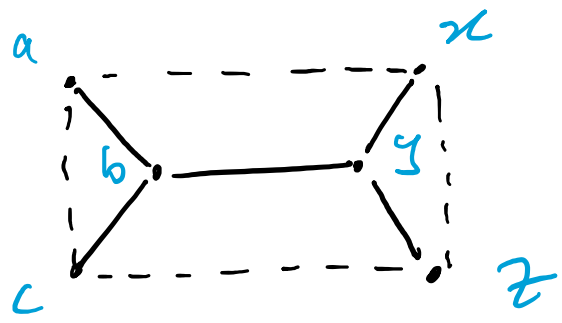


F



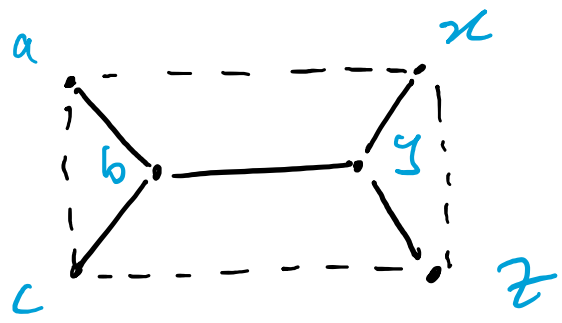
Correctness: Greedy Swap Lemma

- Lem: Let T be a spanning tree and e be an edge not in T. Then,
  - $T \cup \{e\}$ has exactly one cycle C
  - (Swap) For every edge $e' \in C$, the edges $T \cup \{e\} \setminus \{e'\}$ is a spanning tree



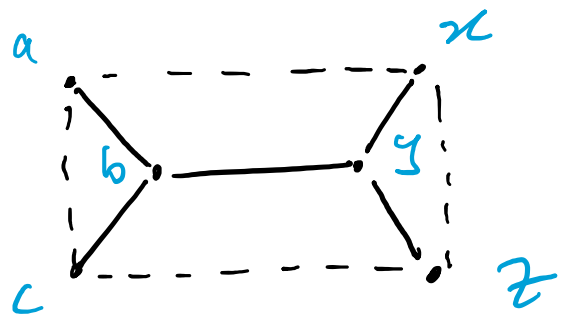
Correctness: Greedy Swap Lemma

- Lem: Let T be a spanning tree and e be an edge not in T. Then,
  - $T \cup \{e\}$ has exactly one cycle C ✅
  - (Swap) For every edge $e' \in C$, the edges $T \cup \{e\} \setminus \{e'\}$ is a spanning tree



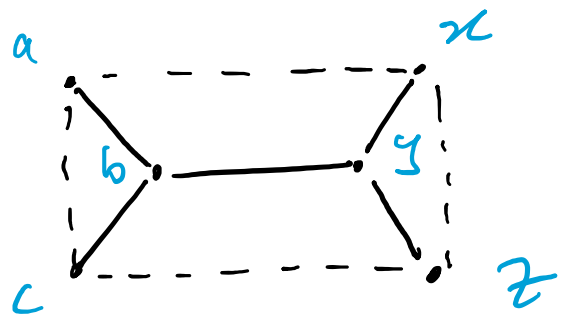
Correctness: Greedy Swap Lemma

- Lem: Let T be a spanning tree and e be an edge not in T. Then,
  - $T \cup \{e\}$ has exactly one cycle C ✅
  - (Swap) For every edge $e' \in C$, the edges $T \cup \{e\} \setminus \{e'\}$ is a spanning tree



Correctness: Greedy Swap Lemma

- Lem: Let T be a spanning tree and e be an edge not in T. Then,
  - $T \cup \{e\}$ has exactly one cycle C ✅
  - (Swap) For every edge $e' \in C$, the edges $T \cup \{e\} \setminus \{e'\}$ is a spanning tree ✅

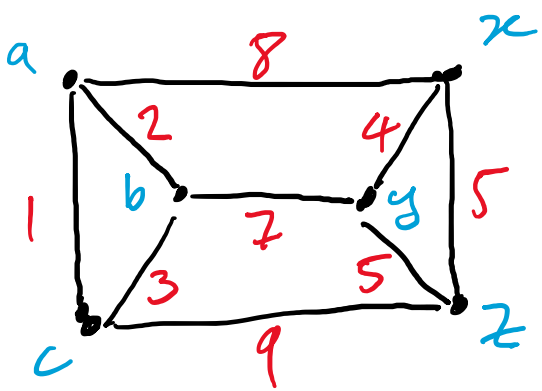


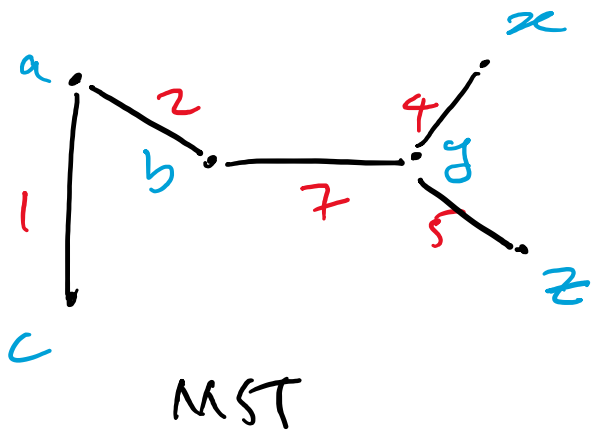
Correctness: Main Lemma

- Let F_i be F and S_i be S at end of iteration i.
- F_0 is empty set and $S_0 = E$.

Lem: After every iteration i of Kruskal's, there is a MST T_i such that

- T_i contains F_i
- $T_i \setminus F_i$ is contained in S_i, i.e. has not yet been discarded by Kruskal's





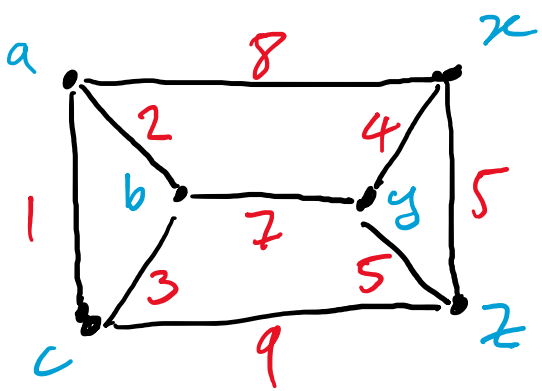
Demo

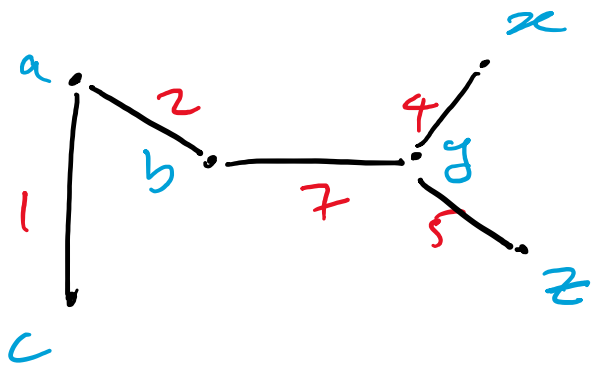
Correctness: Main Lemma

- Let F_i be F and S_i be S at end of iteration i.
- F_0 is empty set and $S_0 = E$.

Lem: After every iteration i of Kruskal's, there is a MST T_i such that

- T_i contains F_i
- $T_i \setminus F_i$ is contained in S_i, i.e. has not yet been discarded by Kruskal's





MST

Qn: How can we use lemma to prove optimality of Kruskal's?

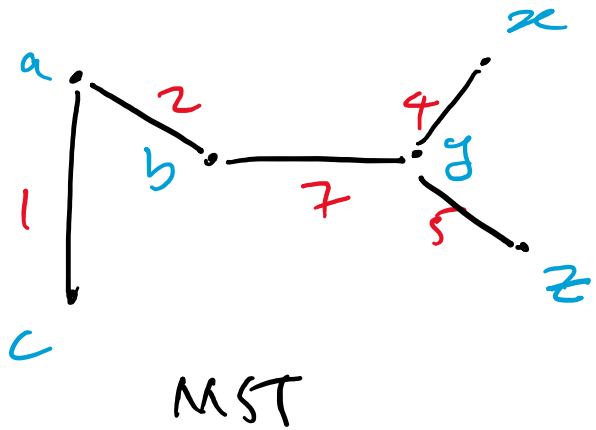
Correctness: Main Lemma

- Let F_i be F and S_i be S at end of iteration i.
- F_0 is empty set and $S_0 = E$.

Lem: After every iteration i of Kruskal's, there is a MST T_i such that

- T_i contains F_i
- $T_i \setminus F_i$ is contained in S_i, i.e. has not yet been discarded by Kruskal's

Pf: By induction on i. Base case $i = 0$



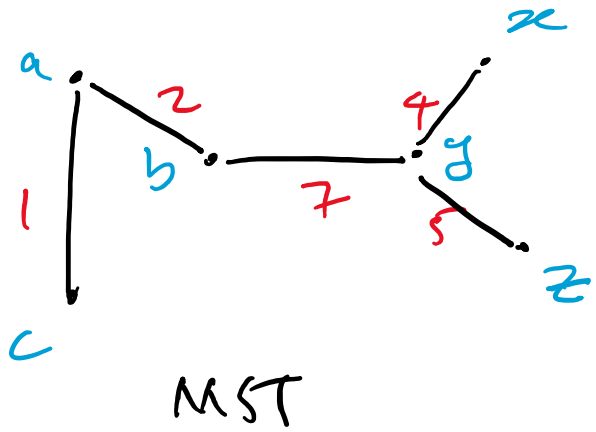
Correctness: Main Lemma

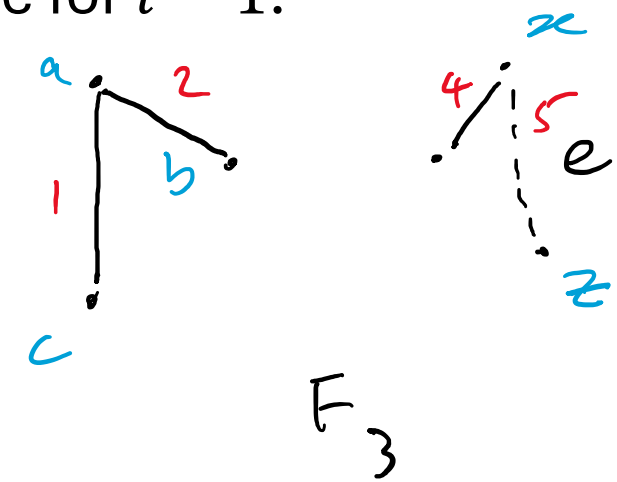
- Let F_i be F and S_i be S at end of iteration i.
- F_0 is empty set and $S_0 = E$.

Lem: After every iteration i of Kruskal's, there is a MST T_i such that

- T_i contains F_i
- $T_i \setminus F_i$ is contained in S_i, i.e. has not yet been discarded by Kruskal's

Pf: By induction on i. Suppose true for $i - 1$.





Running Time Analysis

- Initialize S = E, sorted in increasing order of weight
- Initialize F = empty set
- While F is not spanning:
  - Remove min-weight edge e from S
  - Add e to F if doing so does not create cycles and discard otherwise